

acia6850_device: SR_RDRF is never cleared after an overrun, permanently

freezing the receiver once triggered

Summary

`acia6850_device::data_r()` (`src/devices/machine/6850acia.cpp`) has an asymmetric overrun-handling path that can permanently "stick" the receiver: once an overrun condition occurs, `SR_RDRF` (Receive Data Register Full) is never cleared again, `RDR` freezes at its last value, and every subsequently-arriving character is silently discarded. This requires no unusual configuration to trigger - only that the CPU falls behind the incoming data stream by roughly one character's worth of time, which is easy to hit under any real interrupt-driven RX handler doing nontrivial per-character work (e.g. echoing input back out).

The bug, in the device's own source

`data_r()` currently reads:

```
uint8_t acia6850_device::data_r()
{
    if (!machine().side_effects_disabled())
    {
        if (m_overrun_pending)
        {
            m_status |= SR_OVRN;
            m_overrun_pending = false;
        }
        else
        {
            m_status &= ~SR_OVRN;
            m_status &= ~SR_RDRF;
        }
        ...
        update_irq();
    }
    return m_rdr;
}
```

`SR_RDRF` is cleared **only** in the `else` branch - the one taken when `m_overrun_pending` was already `false` at the time of the read. When an overrun has occurred, the first read after it takes the `if` branch instead: it correctly reports `SR_OVRN` and clears `m_overrun_pending`, but does **not** clear `SR_RDRF`.

This matters because of how overrun itself gets set, in `write_rxc()`:

```
if (m_status & SR_RDRF)
{
    m_overrun_pending = true;
}
else
{
    // ...normal reception: decode the byte into m_rdr, set SR_RDRF...
}
```

A newly-arrived character only gets decoded into `m_rdr` if `SR_RDRF` is currently clear. If it's still set, the new character is discarded and `m_overrun_pending` is set instead - `m_rdr` is left completely unchanged.

Put together: once `SR_RDRF` sticks (from any single overrun), every following character that arrives while it's still set repeats the same `m_overrun_pending = true` path, discarding its own data and leaving `SR_RDRF` set again. The `else` branch in `data_r()` that would finally clear `SR_RDRF` is never reached, because by the time the CPU issues its next read, `m_overrun_pending` has already been re-armed by the next arriving character. `RDR` is now permanently frozen at whatever byte it held at the moment of the very first overrun, and the receiver never recovers without an explicit master reset via `control_w()` (the only other place `m_rx_state` /status get force-reset).

Reproduction

No specific driver should be required - this is reachable from the device model directly:

1. Configure the ACIA for RX-interrupt-enabled operation with a real baud-derived RX clock (`write_rxc` toggling normally).
2. Feed it a burst of characters over RXD fast enough, or hold off the CPU's own read of `ACIADR` long enough, that a second character completes reception before the first is read - triggering exactly one overrun.
3. From that point on, `data_r()` keeps returning the same, frozen byte for every subsequent read, regardless of what's actually being sent on RXD, until the ACIA is explicitly master-reset via `control_w()` (`CR0-CR1 = 3`).

This was found and traced independently in a from-scratch 6809 Forth system, via a from-scratch MAME machine driver (`mecb6809`) using `ACIA6850` for its console UART, while debugging pasted-text input being silently truncated to a single repeated character under interrupt-driven I/O specifically (never observed in polled mode, where this overrun path is naturally avoided since nothing else contends for CPU time between reads).

Suggested fix

`SR_RDRF` should be cleared on every `data_r()` call, once its data has genuinely been consumed, regardless of whether that particular read also happens to be reporting a stale overrun flag from an earlier event:

```
uint8_t acia6850_device::data_r()
{
    if (!machine().side_effects_disabled())
    {
        if (m_overrun_pending)
        {
            m_status |= SR_OVRN;
            m_overrun_pending = false;
        }
        else
        {
            m_status &= ~SR_OVRN;
        }
        m_status &= ~SR_RDRF;
        ...
        update_irq();
    }
    return m_rdr;
}
```

(i.e. hoist `m_status &= ~SR_RDRF;` out of the `else` branch so it runs unconditionally on every read with side effects enabled, while `SR_OVRN` itself keeps its existing, correct, sticky-until-explicitly- cleared-by-the-overrun-branch behavior.)

This preserves the existing overrun *reporting* semantics (`SR_OVRN` still correctly reflects that data was lost) while fixing the receiver so it can actually recover and continue receiving new data afterward, rather than freezing permanently on the first overrun any interrupt-driven consumer happens to hit.

Environment

- MAME version/build: (fill in - not something I have visibility into from here)
- Driver: `mecb6809` (a from-scratch, not-yet-upstreamed driver), but the bug itself is in the shared `acia6850_device` model and should be reproducible from any driver using `ACIA6850` under equivalent timing pressure.